Assignment - 1

(Fitting and Error Analysis)

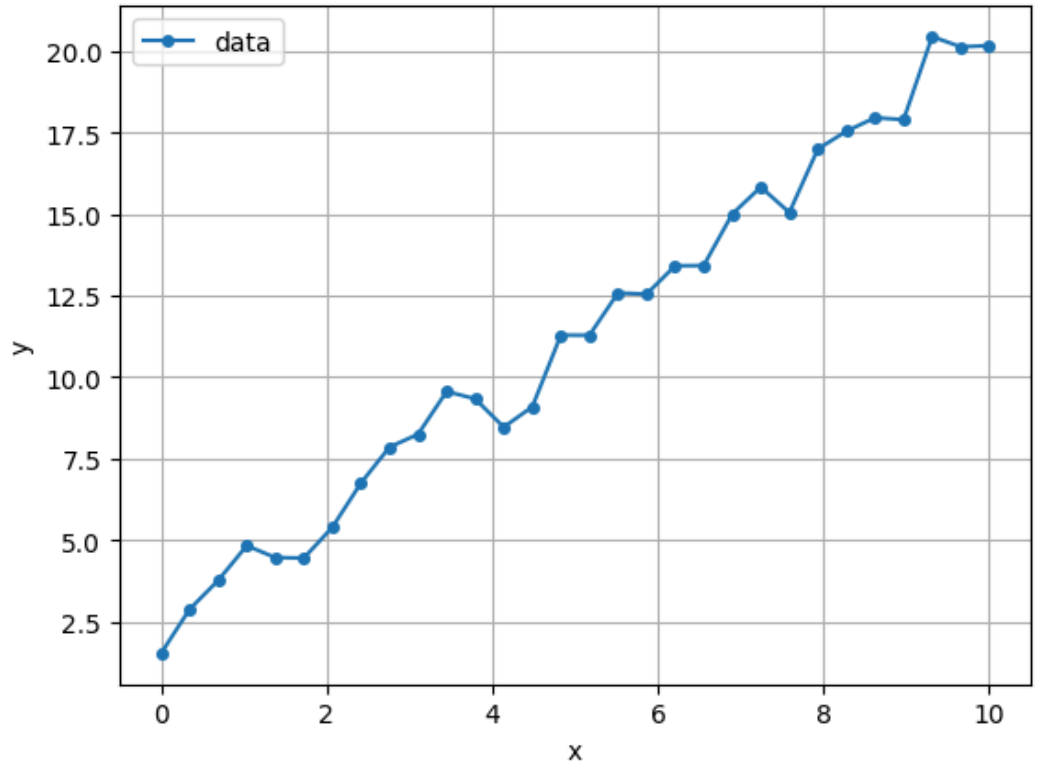
Sanjeev K

Ph25mscst11026

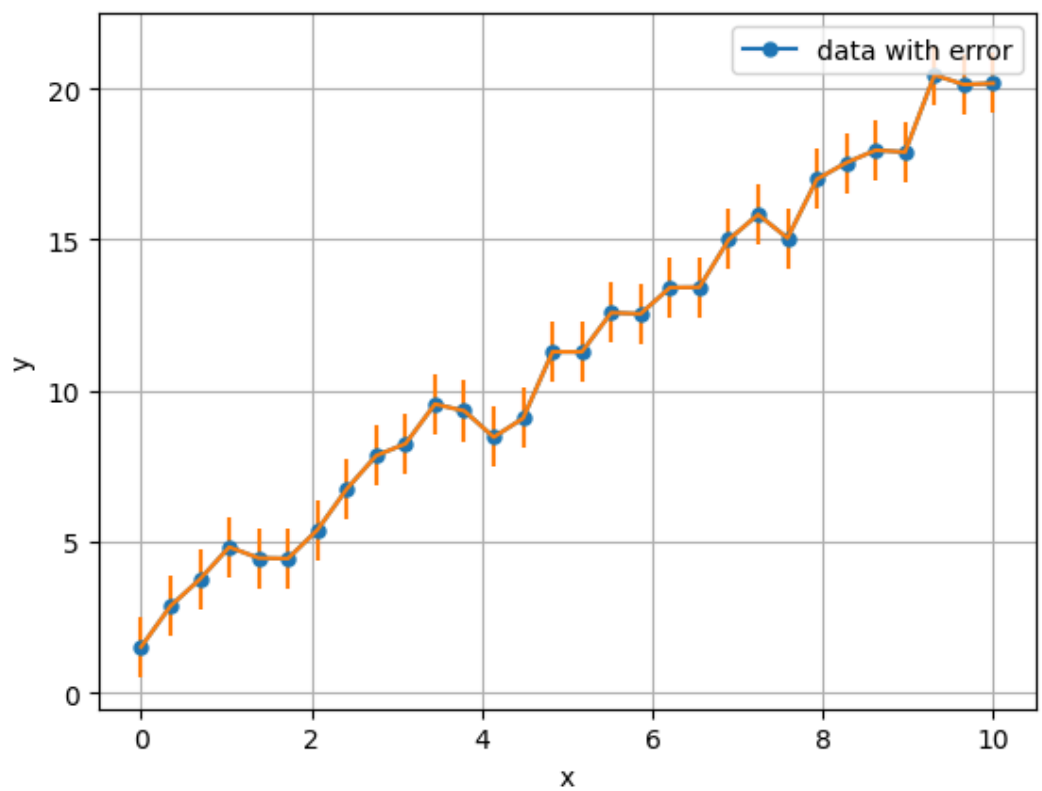
#Read data and plot x Vs y with error

```
import numpy as np
import matplotlib.pyplot as plt

#load and read the data
data = np.loadtxt('/content/data_with_errors.txt')

#split the data into x, y, err(error)
x = data[:, 0]
y = data[:, 1]
err = data[:, 2]

n = len(x) #total number of data points

#plot x vs y with error
plt.plot(x, y, marker='o', markersize=4, label='data with error')
plt.errorbar(x, y, yerr=err) #graph with error
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```

#By observing the graph, we can see that the overall trend of the data points is almost along a straight line. Therefore, this data can be fitted using a straight-line function.




```
#Data with error
```



#Performing weighted least-squares fit

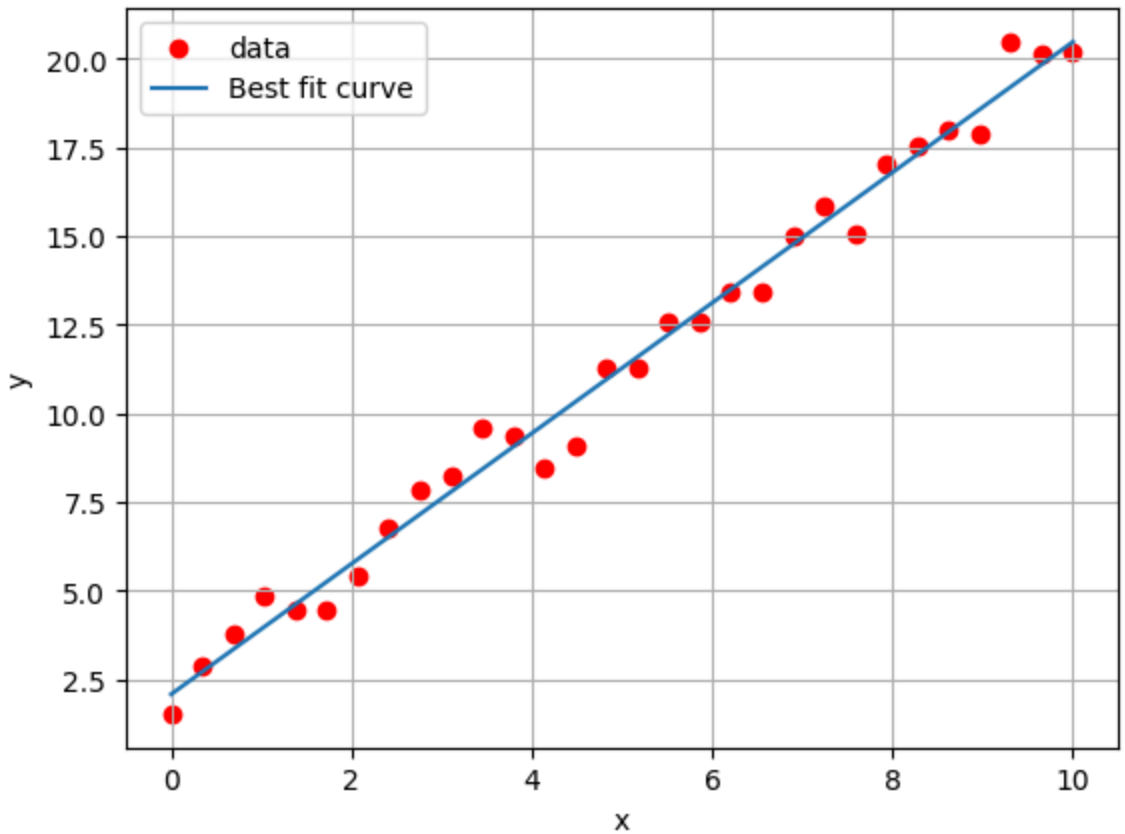
```
#weights
w = 1 / err ** 2

#required sum for the formulae
sum_w   = np.sum(w)
sum_x  = np.sum(w * x)
sum_y  = np.sum(w * y)
sum_xx = np.sum(w * x**2)
sum_xy = np.sum(w * x * y)

#Least squares formulae
m = (sum_w * sum_xy - sum_x * sum_y) / (sum_w * sum_xx - sum_x**2)
c = (sum_xx * sum_y - sum_x * sum_xy) / (sum_w * sum_xx - sum_x**2)

#Calculating parameters of the function
print(f'slope: {m}')   #1.839410913512797
print(f'intercept: {c}')  #2.075294232436016
```

Output:

```
slope: 1.839410913512797
intercept: 2.075294232436016
```

#Plotting the best fit curve and the data together.

```
#best fit line
y_fit = m * x + c
plt.scatter(x, y, color='red', label='data')
plt.plot(x, y_fit, label='Best fit curve')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```



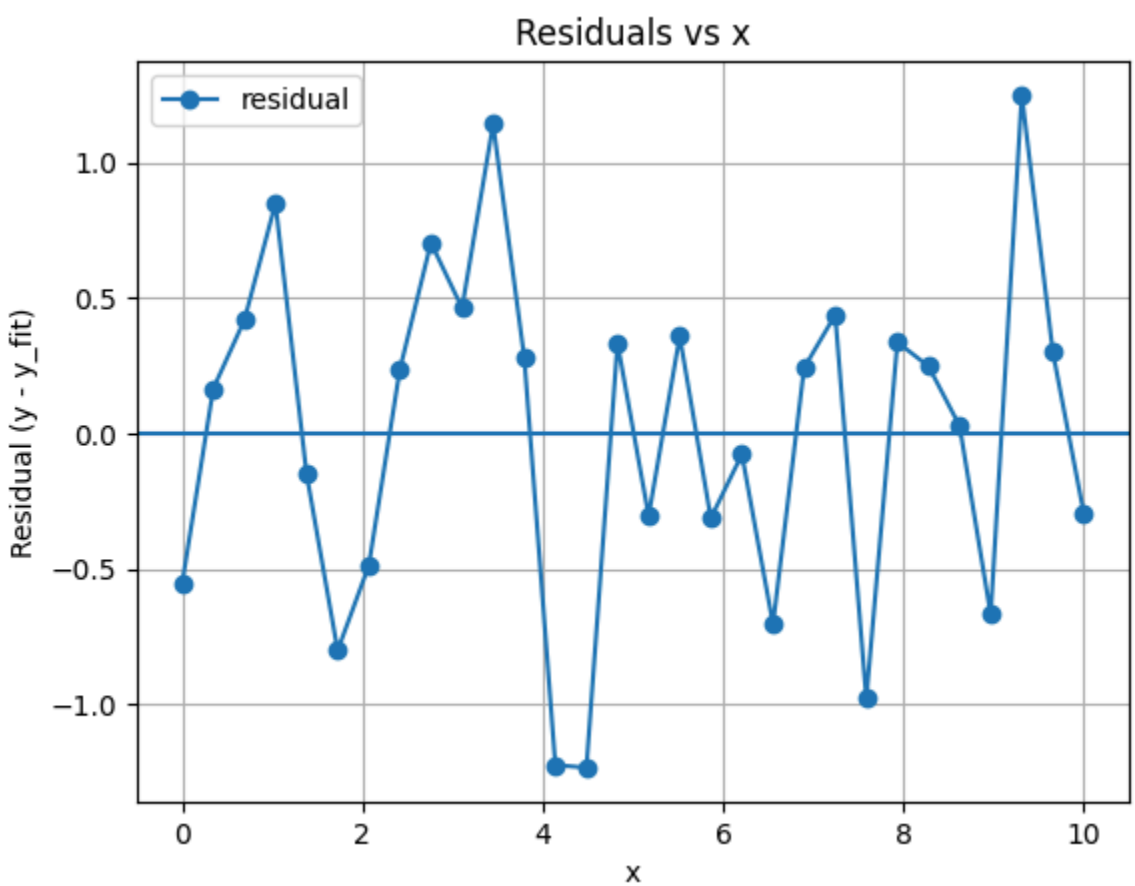
#Calculate the residual and plot it Vs 'x'

```
# Residuals
residual = y - y_fit

# Plot residuals vs x
plt.figure()
plt.plot(x, residual, marker='o', label='residual')
plt.axhline(0)   # reference line
plt.xlabel("x")
plt.ylabel("Residual (y - y_fit)")
plt.title("Residuals vs x")
plt.legend()
plt.grid(True)
plt.show()
```

```python
# Print Residuals
print(f'residuals: {residual}')
```

Output:

```
residuals: [-0.55242123  0.16530538  0.41778983  0.84853545 -0.1489921
-0.80009349 -0.48747987  0.23559058  0.70316319  0.46264065  1.14077126
0.28063371 -1.22212167 -1.23219006  0.33268539 -0.30391599  0.35826746
-0.31247393 -0.07709731 -0.70339486  0.24170975  0.43733721 -0.97788818
0.33646427 0.24783889  0.0277805  -0.66768505  1.24800657  0.29915902
-0.29792537]
```



#Straight-line fit using SciPy

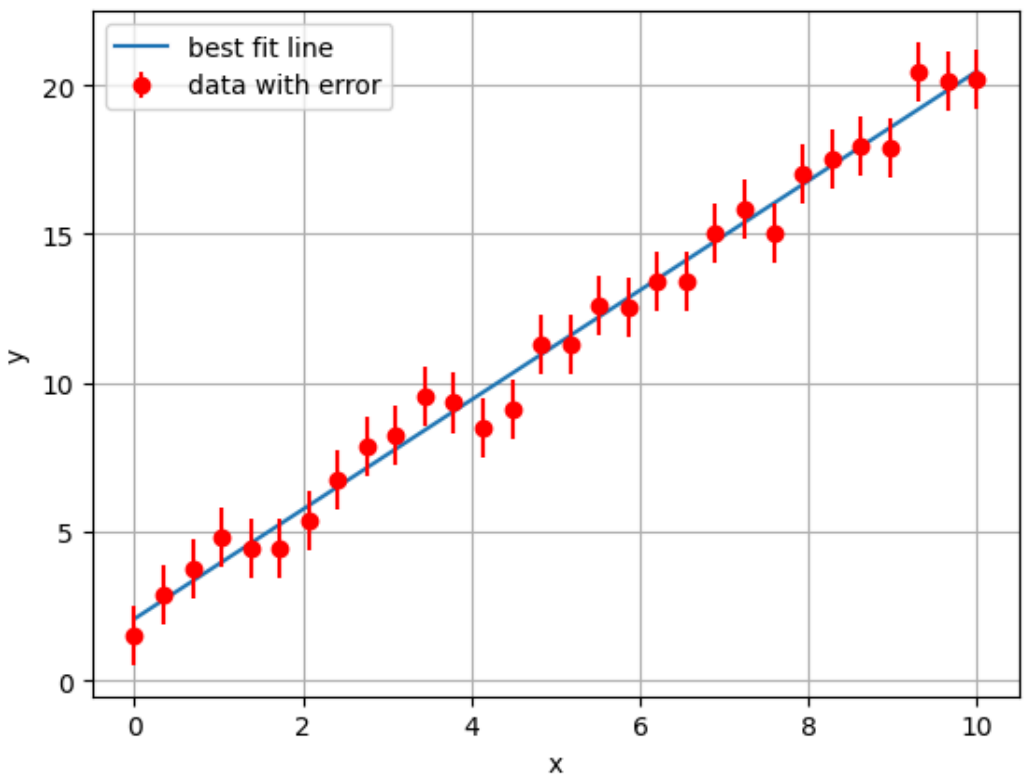
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

#Define function
def straight_line(x, m, c):
    return  m * x + c

#Load data
x, y, err = np.loadtxt("data_with_errors.txt", unpack=True) #it read the
data columnwise

#Weighted curve fit using scipy
popt, pcov = curve_fit(straight_line, x, y, sigma=err,
absolute_sigma=True)
m, c = popt #extracting slope and error
sigma_m, sigma_c = np.sqrt(np.diag(pcov)) #uncertainity in slope and
intercept

print(f'slope: {m}')
print(f'intercept: {c}')
print(f'error in slope: {sigma_a}')
print(f'error in intercept: {sigma_b}')
```

Output:

```
slope: 1.83941091134803
intercept: 2.075294235337993

error in slope: 0.3562046133014505
error in intercept: 0.0611713629277751
```

```
#plot with error
```

```
plt.plot(x, straight_line(x, m, c), label='best fit line')
plt.errorbar(x, y, yerr=err, fmt='o',color='red', label='data with error')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```



I really enjoyed doing this assignment:)